

Requirement 4 Baseline Fix: Practical Code Walkthrough

OLA Project

1 What Was Wrong in Practice

The learners were not necessarily the source of the almost-linear regret. The practical issue was that the run loop used only the dynamic realised clairvoyant. That oracle knows the realised highest competing bids for every round before choosing the action. This is useful as a prophet upper bound, but too strong as the main learning-regret baseline.

2 Old Code: Prophet Baseline Only

This is the problematic pattern in `run_nonstationary_trials`: one scalar oracle value is computed from realised `env.m`, then subtracted from the learner utility at every round.

```
1 if clairvoyant_cache is not None and i in clairvoyant_cache:
2     trial_opt = float(clairvoyant_cache[i]["opt_per_round"])
3 else:
4     _, trial_opt = compute_clairvoyant_dynamic_multi(
5         m_seq=env.m, values=env.values, bid_sets=env.bid_sets,
6         budget=env.budget, conflict_edges=env.conflict_edges,
7     )
8
9 ...
10
11 regret_per_trial.append(np.cumsum(trial_opt - utilities))
```

Line	Practical meaning
1–2	If a cached oracle exists, load its per-round value. This is fast, but it keeps using the same prophet definition.
4–7	Otherwise solve a dynamic LP on <code>env.m</code> . This gives the oracle access to the realised competing bids of all rounds.
11	The learner is compared against <code>trial_opt</code> at every round. Any prophet information premium becomes a roughly constant regret slope.

3 New Code Part 1: Store the True Block Distributions

The shock environment already samples each stationary block from a Beta distribution. The fix stores the selected Beta parameters while generating the same sampled sequence. No extra random draw is introduced.

```
1 def _draw(self, seed):
2     rng = np.random.default_rng(seed)
3     self.shock_blocks = None
4     if self.mode == "drift":
5         self.m = self._build_drift(rng)
6     else:
7         self.m = self._build_shocks(rng)
8
9 def _build_shocks(self, rng):
10     ...
11     self.shock_blocks = []
12     for i in range(self.N):
13         campaign_blocks = []
14         for b in range(n_blocks):
15             start = b * self.block_size
16             end = min(start + self.block_size, T)
17             a, bb = regimes[rng.integers(0, self.n_regimes)]
18             campaign_blocks.append((start, end, float(a), float(bb)))
19             m[i, start:end] = rng.beta(a, bb, size=end - start)
20         self.shock_blocks.append(campaign_blocks)
21     return m
```

Line	Practical meaning
3	Reset block metadata whenever the environment is redrawn.
11	Prepare one metadata list per campaign.
14–17	Keep the exact interval boundaries and the exact Beta parameters selected for that campaign and block.
18	Sample the realised competing bids exactly as before.
19	Save the metadata after the samples are generated.

4 New Code Part 2: Convert Blocks into Expected Win Probabilities

The expected clairvoyant needs $P(b \geq m)$, not the realised m . For a Beta block, this is the Beta CDF evaluated at each bid.

```

1 def piecewise_win_probabilities(self):
2     if self.mode != "shocks" or self.shock_blocks is None:
3         raise ValueError("piecewise_win_probabilities is only available for mode='shocks'.")
4
5     from scipy.stats import beta
6
7     n_blocks = len(self.shock_blocks[0])
8     out = []
9     for b in range(n_blocks):
10        start, end = self.shock_blocks[0][b][:2]
11        block_probs = []
12        for i in range(self.N):
13            i_start, i_end, alpha, beta_param = self.shock_blocks[i][b]
14            if (i_start, i_end) != (start, end):
15                raise RuntimeError("Shock block boundaries are inconsistent across campaigns.")
16            block_probs.append(beta.cdf(self.bid_sets[i], alpha, beta_param))
17        out.append((start, end, block_probs))
18    return out

```

Line	Practical meaning
2–3	Fail explicitly if someone tries to use this baseline outside the shock setting.
5	Use the analytical Beta distribution instead of sampled realised bids.
10	Use common block boundaries.
13–16	For each campaign, compute the expected win probability of every bid in the block.
17–18	Return one expected distributional snapshot per block.

5 New Code Part 3: Solve the Piecewise Expected Oracle

The new oracle solves one LP over blocks. It may know block distributions and changepoints, but not round-by-round realised bids.

```

1 def compute_piecewise_expected_clairvoyant(env):
2     blocks = env.piecewise_win_probabilities()
3     actions = _build_feasible_joint_actions(env.bid_sets, env.conflict_edges)
4     n_blocks = len(blocks)
5     n_actions = len(actions)
6     n_vars = n_blocks * n_actions
7
8     f = np.zeros(n_vars)
9     c = np.zeros(n_vars)
10    lengths = np.zeros(n_blocks, dtype=float)
11
12    for s, (start, end, win_prob_list) in enumerate(blocks):
13        length = end - start
14        lengths[s] = length
15        for a_idx, action in enumerate(actions):
16            var_idx = s * n_actions + a_idx
17            for i, k in enumerate(action):
18                if k >= 0:
19                    p_win = win_prob_list[i][k]
20                    bid = env.bid_sets[i][k]
21                    f[var_idx] += length * (env.values[i] - bid) * p_win
22                    c[var_idx] += length * bid * p_win

```

Line	Practical meaning
2	Read expected block probabilities, not realised samples.
3	Enumerate only feasible joint actions, including abstention and conflict constraints.
8–10	Allocate expected utility, expected cost, and block lengths.
12–22	Fill the LP coefficients: expected utility and expected spend of each action in each block, multiplied by block length.

```

1 A_eq = np.zeros((n_blocks, n_vars))
2 for s in range(n_blocks):
3     A_eq[s, s * n_actions:(s + 1) * n_actions] = 1.0
4
5 res = optimize.linprog(
6     -f,
7     A_ub=np.array([c]),
8     b_ub=np.array([env.budget]),
9     A_eq=A_eq,
10    b_eq=np.ones(n_blocks),
11    bounds=[(0.0, 1.0)] * n_vars,
12    method="highs",
13 )

```

Line	Practical meaning
1–3	Force each block to choose a probability distribution over actions.
5–13	Maximize expected utility subject to the original total budget B . The budget is still shared across the whole horizon.

6 New Code Part 4: Save Both Regrets

The final change keeps the old prophet regret and adds the new practical diagnostic regret.

```

1 piecewise_expected = None
2 if hasattr(env, "piecewise_win_probabilities"):
3     try:
4         _, piecewise_expected = compute_piecewise_expected_clairvoyant(env)
5     except ValueError:
6         piecewise_expected = None
7
8 ...
9
10 regret_per_trial.append(np.cumsum(trial_opt - utilities))
11 if piecewise_expected is not None:
12     regret_piecewise_per_trial.append(np.cumsum(piecewise_expected - utilities))

```

Line	Practical meaning
1–6	Try to compute the new baseline only when the environment supports it.
10	Preserve the old dynamic realised oracle as a prophet comparison.
11–12	Add the corrected regret curve against the piecewise expected oracle. Same learner utilities, different baseline.

7 Expected Effect

After this change, `req4_regret.png` still shows regret against the prophet oracle. The new piecewise-expected regret plot, saved as `req4_regret_piecewise_expected.png`, should be the relevant plot for diagnosing learner performance. If the theory is right, the new regret should be much less linearly dominated, because the artificial round-by-round information premium has been removed.